COMPLETE SOURCE CODE EXPLANATION

Utilities, Exceptions, Data Structures & Main

Line-by-Line Explanation of File I/O, Error Hierarchy & Core Runtime

Covers File Handling (ActivityLogger, BillReceiptWriter, ReportExporter), Custom Exception Hierarchy, DSA structures (Queue, Sort/Search), and Main.java application bootstrapping.

Project: Restaurant Management System

Language: Java 17 / JavaFX / JDBC

Scope: Detailed Line-by-Line Breakdown

Table of Contents

ActivityLogger.java	src\main\java\com\restaurant\util\ActivityLogger.java
AlertUtil.java	src\main\java\com\restaurant\util\AlertUtil.java
BillReceiptWriter.java	src\main\java\com\restaurant\util\BillReceiptWriter.java
CurrencyUtil.java	src\main\java\com\restaurant\util\CurrencyUtil.java
DateTimeUtil.java	src\main\java\com\restaurant\util\DateTimeUtil.java
PasswordUtil.java	src\main\java\com\restaurant\util\PasswordUtil.java
ReportExporter.java	src\main\java\com\restaurant\util\ReportExporter.java
ValidationUtil.java	src\main\java\com\restaurant\util\ValidationUtil.java
DatabaseOperationException.java	src\main\java\com\restaurant\exception\DatabaseOperationException.java
InsufficientStockException.java	src\main\java\com\restaurant\exception\InsufficientStockException.java
InvalidLoginException.java	src\main\java\com\restaurant\exception\InvalidLoginException.java
InvalidOrderException.java	src\main\java\com\restaurant\exception\InvalidOrderException.java
InvalidReservationException.java	src\main\java\com\restaurant\exception\InvalidReservationException.java
MenuItemNotAvailableException.java	src\main\java\com\restaurant\exception\MenuItemNotAvailableException.java
PaymentFailedException.java	src\main\java\com\restaurant\exception\PaymentFailedException.java
TableNotAvailableException.java	src\main\java\com\restaurant\exception\TableNotAvailableException.java
DataStructureManager.java	src\main\java\com\restaurant\data\DataStructureManager.java
OrderQueue.java	src\main\java\com\restaurant\data\OrderQueue.java
SearchSortUtil.java	src\main\java\com\restaurant\data\SearchSortUtil.java
Main.java	src\main\java\com\restaurant\Main.java

ActivityLogger.java

Path: src\main\java\com\restaurant\util\ActivityLogger.java | Total Lines: 133 | Logical Blocks: 12

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Disk I/O File Handling engine. Handles formatted file outputs, thread-safe append streams, and CSV/text report generation.

Lines 1–23 Package & Library Imports

```
1  package com.restaurant.util;
2
3  import java.io.BufferedWriter;
4  import java.io.FileWriter;
5  import java.io.IOException;
6  import java.nio.file.Files;
7  import java.nio.file.Path;
8  import java.nio.file.Paths;
9  import java.time.LocalDateTime;
10 import java.time.format.DateTimeFormatter;
11 import java.util.logging.Level;
12 import java.util.logging.Logger;
13
14 /**
15  * FILE HANDLING — Activity Logger
16  *
17  * Demonstrates Java file I/O using FileWriter and BufferedWriter.
18  * Appends timestamped log entries to logs/activity.log for every
19  * significant application event (login, logout, order, payment, error).
20  *
21  * Thread-safe: uses synchronized append to prevent interleaved writes
22  * when multiple threads call log() concurrently.
23  */
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Concurrency Support	Multi-threading utilities (BlockingQueue, Executors, Atomic primitives) used for asynchronous tasks.
File Handling I/O	File streams and writers (FileWriter, BufferedWriter, PrintWriter) for receipts and log generation.
Line Analysis	Covers lines 1 to 23. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 24–24 Class Declaration & Heritage

```
24  public class ActivityLogger {
```

Architecture Role	Defines ActivityLogger as a core component in the Restaurant Management System.
Utility / I/O Engine	Provides reusable, thread-safe helper services across logging, receipt serialization, and date-time arithmetic.
Access & Modifiers	Declared with public visibility on line 24, allowing cross-package invocation.

Lines 25–52 Class Members & Implementation

```
25
26      private static final Logger LOGGER = Logger.getLogger(ActivityLogger.class.getName());
27      private static final String LOG_DIR  = "logs";
28      private static final String LOG_FILE = "logs/activity.log";
29      private static final DateTimeFormatter FORMATTER =
30              DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");
31
32      // Singleton instance
33      private static ActivityLogger instance;
34
35      // Private constructor — ensures log directory exists on first use
36      private ActivityLogger() {
37          ensureLogDirectoryExists();
38      }
39
40      /** Returns the single shared instance (lazy-initialised, thread-safe). */
41      public static synchronized ActivityLogger getInstance() {
42          if (instance == null) {
43              instance = new ActivityLogger();
44          }
45          return instance;
46      }
47
48      // ------------------------------------------------------------------
49      // Public logging API
50      // ------------------------------------------------------------------
51
52      /** Log a successful user login. */
```

Component	Class Members & Implementation
Thread Synchronization	Guarantees thread-safety: prevents race conditions by allowing only one thread to execute this critical block at any instant.
Line-by-Line Breakdown	Lines 25–52 (28 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 53–57 Method: logLogin()

```
53      public void logLogin(String username, String role) {
54          writeEntry("LOGIN", "User '" + username + "' logged in as " + role);
55      }
56
57      /** Log a user logout event. */
```

Component	Method: logLogin()
Line-by-Line Breakdown	Lines 53–57 (5 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 58–62 Method: logLogout()

```
58      public void logLogout(String username) {
59          writeEntry("LOGOUT", "User '" + username + "' logged out");
60      }
61
62      /** Log a new order being placed. */
```

Component	Method: logLogout()
Line-by-Line Breakdown	Lines 58–62 (5 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 63–68 Method: logOrderCreated()

```
63      public void logOrderCreated(String orderNumber, String tableNumber, double total) {
64          writeEntry("ORDER_CREATED",
65                  String.format("Order %s | Table: %s | Total: %.2f", orderNumber, tableNumber, total));
66      }
67
68      /** Log a payment being completed. */
```

Component	Method: logOrderCreated()
Line-by-Line Breakdown	Lines 63–68 (6 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 69–74 Method: logPayment()

```
69      public void logPayment(String orderNumber, double amount, String method) {
70          writeEntry("PAYMENT",
71                  String.format("Order %s | Amount: %.2f | Method: %s", orderNumber, amount, method));
72      }
73
74      /** Log a bill/receipt being saved to disk. */
```

Component	Method: logPayment()
Line-by-Line Breakdown	Lines 69–74 (6 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 75–80 Method: logReceiptSaved()

```
75      public void logReceiptSaved(String orderNumber, String filePath) {
76          writeEntry("RECEIPT_SAVED",
77                  String.format("Order %s | File: %s", orderNumber, filePath));
78      }
79
80      /** Log a report being exported. */
```

Component	Method: logReceiptSaved()
File Handling Logic	Interacts with the disk filesystem: ensures parent directory existence, opens output streams, and writes formatted text or CSV records.
Line-by-Line Breakdown	Lines 75–80 (6 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 81–86 Method: logReportExported()

```
81      public void logReportExported(String reportType, String filePath) {
82          writeEntry("REPORT_EXPORTED",
83                  String.format("Type: %s | File: %s", reportType, filePath));
84      }
85
86      /** Log any application-level error with the exception message. */
```

Component	Method: logReportExported()
File Handling Logic	Interacts with the disk filesystem: ensures parent directory existence, opens output streams, and writes formatted text or CSV records.
Line-by-Line Breakdown	Lines 81–86 (6 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 87–91 Method: logError()

```
87      public void logError(String context, String errorMessage) {
88          writeEntry("ERROR", context + " | " + errorMessage);
89      }
90
91      /** Log a generic informational event. */
```

Component	Method: logError()
Line-by-Line Breakdown	Lines 87–91 (5 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 92–121 Method: logInfo()

```
92      public void logInfo(String message) {
93          writeEntry("INFO", message);
94      }
95
96      // ------------------------------------------------------------------
97      // Core file-writing logic (FILE HANDLING)
98      // ------------------------------------------------------------------
99
100     /**
101      * Appends a formatted log entry to the activity log file.
102      * Uses try-with-resources to guarantee the BufferedWriter is closed
103      * even if an IOException occurs — demonstrating proper file handling.
104      *
105      * Marked synchronized so concurrent threads don't interleave lines.
106      */
107     private synchronized void writeEntry(String eventType, String message) {
108         String timestamp = LocalDateTime.now().format(FORMATTER);
109         String logLine   = String.format("[%s] [%-16s] %s%n", timestamp, eventType, message);
110
111         // try-with-resources: FileWriter(path, true) → append mode
112         try (BufferedWriter writer = new BufferedWriter(new FileWriter(LOG_FILE, true))) {
113             writer.write(logLine);
114             writer.flush(); // ensure the OS buffer is flushed immediately
115         } catch (IOException e) {
116             // Use java.util.logging as fallback — never silently swallow errors
117             LOGGER.log(Level.WARNING, "ActivityLogger: failed to write to " + LOG_FILE, e);
118         }
119     }
120
121     /** Creates the logs/ directory if it does not already exist. */
```

Component	Method: logInfo()
File Handling Logic	Interacts with the disk filesystem: ensures parent directory existence, opens output streams, and writes formatted text or CSV records.
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Thread Synchronization	Guarantees thread-safety: prevents race conditions by allowing only one thread to execute this critical block at any instant.
Line-by-Line Breakdown	Lines 92–121 (30 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 122–133` **Method: ensureLogDirectoryExists()**

```
122       private void ensureLogDirectoryExists() {
123           try {
124               Path dir = Paths.get(LOG_DIR);
125               if (!Files.exists(dir)) {
126                   Files.createDirectories(dir);
127                   LOGGER.info("ActivityLogger: created log directory → " + dir.toAbsolutePath());
128               }
129           } catch (IOException e) {
130               LOGGER.log(Level.WARNING, "ActivityLogger: could not create log directory", e);
131           }
132       }
133   }
```

Component	Method: ensureLogDirectoryExists()
File Handling Logic	Interacts with the disk filesystem: ensures parent directory existence, opens output streams, and writes formatted text or CSV records.
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 122–133 (12 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

AlertUtil.java

Path: src\main\java\com\restaurant\util\AlertUtil.java | Total Lines: 45 | Logical Blocks: 7

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the AlertUtil role in the Restaurant Management architecture.

Lines 1–10 Package & Library Imports

```
1  package com.restaurant.util;
2
3  import javafx.scene.control.Alert;
4  import javafx.scene.control.ButtonType;
5
6  import java.util.Optional;
7
8  /**
9   * Utility for displaying modern JavaFX dialogs and alerts.
10  */
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
JavaFX Dependency	Binds FXML controls (TableViews, Buttons, ComboBoxes, Labels) to enable real-time UI interaction.
Line Analysis	Covers lines 1 to 10. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 11–11 Class Declaration & Heritage

```
11 public class AlertUtil {
```

Architecture Role	Defines AlertUtil as a core component in the Restaurant Management System.
Utility / I/O Engine	Provides reusable, thread-safe helper services across logging, receipt serialization, and date-time arithmetic.
Access & Modifiers	Declared with public visibility on line 11, allowing cross-package invocation.

Lines 12–12 Class Members & Implementation

```
12
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 12–12 (1 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 13–20 Method: showInfo()

```
13      public static void showInfo(String title, String message) {
14          Alert alert = new Alert(Alert.AlertType.INFORMATION);
15          alert.setTitle(title);
16          alert.setHeaderText(null);
17          alert.setContentText(message);
18          alert.showAndWait();
19      }
20
```

Component	Method: showInfo()
Line-by-Line Breakdown	Lines 13–20 (8 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 21–28 Method: showWarning()

```
21      public static void showWarning(String title, String message) {
22          Alert alert = new Alert(Alert.AlertType.WARNING);
23          alert.setTitle(title);
24          alert.setHeaderText(null);
25          alert.setContentText(message);
26          alert.showAndWait();
27      }
28
```

Component	Method: showWarning()
Line-by-Line Breakdown	Lines 21–28 (8 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 29–36 Method: showError()

```
29      public static void showError(String title, String message) {
30          Alert alert = new Alert(Alert.AlertType.ERROR);
31          alert.setTitle(title);
32          alert.setHeaderText(null);
33          alert.setContentText(message);
34          alert.showAndWait();
35      }
36
```

Component	Method: showError()
Line-by-Line Breakdown	Lines 29–36 (8 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 37–45` **Method: showConfirmation()**

```
37      public static boolean showConfirmation(String title, String message) {
38          Alert alert = new Alert(Alert.AlertType.CONFIRMATION);
39          alert.setTitle(title);
40          alert.setHeaderText(null);
41          alert.setContentText(message);
42          Optional<ButtonType> result = alert.showAndWait();
43          return result.isPresent() && result.get() == ButtonType.OK;
44      }
45  }
```

Component	Method: showConfirmation()
Line-by-Line Breakdown	Lines 37–45 (9 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

BillReceiptWriter.java

Path: src\main\java\com\restaurant\util\BillReceiptWriter.java | Total Lines: 137 | Logical Blocks: 8

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Disk I/O File Handling engine. Handles formatted file outputs, thread-safe append streams, and CSV/text report generation.

Lines 1–28 Package & Library Imports

```
1   package com.restaurant.util;
2
3   import com.restaurant.model.Order;
4   import com.restaurant.model.OrderItem;
5
6   import java.io.BufferedWriter;
7   import java.io.FileWriter;
8   import java.io.IOException;
9   import java.nio.file.Files;
10  import java.nio.file.Path;
11  import java.nio.file.Paths;
12  import java.time.LocalDateTime;
13  import java.time.format.DateTimeFormatter;
14  import java.util.logging.Level;
15  import java.util.logging.Logger;
16
17  /**
18   * FILE HANDLING — Bill Receipt Writer
19   *
20   * Saves a formatted text receipt for each completed order to the
21   * receipts/ directory.  File name: receipts/RECEIPT_<orderNumber>.txt
22   *
23   * Demonstrates:
24   *  - FileWriter + BufferedWriter for sequential text output
25   *  - try-with-resources to guarantee stream closure
26   *  - Path / Files API (NIO) to create the output directory
27   *  - IOException handling with meaningful error propagation
28   */
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
File Handling I/O	File streams and writers (FileWriter, BufferedWriter, PrintWriter) for receipts and log generation.
Line Analysis	Covers lines 1 to 28. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 29–29 Class Declaration & Heritage

```
29  public class BillReceiptWriter {
```

Architecture Role	Defines BillReceiptWriter as a core component in the Restaurant Management System.
Utility / I/O Engine	Provides reusable, thread-safe helper services across logging, receipt serialization, and date-time arithmetic.
Access & Modifiers	Declared with public visibility on line 29, allowing cross-package invocation.

Lines 30–47 Class Members & Implementation

```
30
31      private static final Logger LOGGER = Logger.getLogger(BillReceiptWriter.class.getName());
32      private static final String RECEIPTS_DIR = "receipts";
33      private static final DateTimeFormatter FORMATTER =
34              DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");
35
36      // ------------------------------------------------------------------------
37      // Public API
38      // ------------------------------------------------------------------------
39
40      /**
41       * Writes the formatted receipt text to a .txt file.
42       *
43       * @param order          the completed order
44       * @param receiptContent the pre-formatted receipt string (from BillingService)
45       * @return the absolute path of the saved file, or null on failure
46       * @throws IOException if the file cannot be created or written
47       */
```

Component	Class Members & Implementation
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 30–47 (18 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 48–71

Method: saveReceipt()

```
48      public String saveReceipt(Order order, String receiptContent) throws IOException {
49          ensureReceiptDirectoryExists();
50
51          String fileName  = buildFileName(order.getOrderNumber());
52          String filePath  = RECEIPTS_DIR + "/" + fileName;
53
54          // try-with-resources ensures BufferedWriter is always closed
55          try (BufferedWriter writer = new BufferedWriter(new FileWriter(filePath, false))) {
56              writer.write(receiptContent);
57              writer.newLine();
58              writer.write("Saved at: " + LocalDateTime.now().format(FORMATTER));
59              writer.newLine();
60              writer.flush();
61          }
62          // IOException propagates to the caller — caller decides how to inform the user
63
64          LOGGER.info("Receipt saved → " + Paths.get(filePath).toAbsolutePath());
65          return Paths.get(filePath).toAbsolutePath().toString();
66      }
67
68      /**
69       * Convenience overload: generates a simple receipt from raw order data
70       * without needing a pre-formatted string.
71       */
```

Component	Method: saveReceipt()
File Handling Logic	Interacts with the disk filesystem: ensures parent directory existence, opens output streams, and writes formatted text or CSV records.
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 48–71 (24 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 72–80

Method: saveSimpleReceipt()

```
72      public String saveSimpleReceipt(Order order) throws IOException {
73          String content = buildSimpleReceipt(order);
74          return saveReceipt(order, content);
75      }
76
77      // ------------------------------------------------------------------------
78      // Internal helpers
79      // ------------------------------------------------------------------------
80
```

Component	Method: saveSimpleReceipt()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 72–80 (9 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 81–86` **Method: buildFileName()**

```
81      private String buildFileName(String orderNumber) {
82          // Sanitise order number so it is safe for use as a filename
83          String safe = orderNumber.replaceAll("[^a-zA-Z0-9_\\-]", "_");
84          return "RECEIPT_" + safe + ".txt";
85      }
86
```

Component	Method: buildFileName()
File Handling Logic	Interacts with the disk filesystem: ensures parent directory existence, opens output streams, and writes formatted text or CSV records.
Line-by-Line Breakdown	Lines 81–86 (6 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 87–129` **Method: buildSimpleReceipt()**

```
87      private String buildSimpleReceipt(Order order) {
88          StringBuilder sb = new StringBuilder();
89          sb.append("================================================\n");
90          sb.append("           GRAND FEAST — FINE DINING            \n");
91          sb.append("================================================\n");
92          sb.append(String.format("Order No  : %s%n",  order.getOrderNumber()));
93          sb.append(String.format("Table     : %s%n",
94                  order.getTableNumber() != null ? order.getTableNumber() : "Takeaway"));
95          sb.append(String.format("Customer  : %s%n",
96                  order.getCustomerName() != null ? order.getCustomerName() : "Walk-in"));
97          sb.append(String.format("Date/Time : %s%n",
98                  order.getOrderDateTime() != null
99                          ? order.getOrderDateTime().format(FORMATTER)
100                         : LocalDateTime.now().format(FORMATTER)));
101         sb.append("------------------------------------------------\n");
102         sb.append(String.format("%-25s %5s %10s%n", "Item", "Qty", "Subtotal"));
103         sb.append("------------------------------------------------\n");
104
105         if (order.getItems() != null) {
106             for (OrderItem item : order.getItems()) {
107                 String name = item.getItemName();
108                 if (name != null && name.length() > 24) name = name.substring(0, 21) + "...";
109                 sb.append(String.format("%-25s %5d %10.2f%n",
110                         name, item.getQuantity(), item.getSubtotal()));
111             }
112         }
113
114         sb.append("------------------------------------------------\n");
115         sb.append(String.format("%-32s : %10.2f%n", "Subtotal",       order.getSubtotal()));
116         sb.append(String.format("%-32s : %10.2f%n", "Tax",            order.getTaxAmount()));
117         sb.append(String.format("%-32s : %10.2f%n", "Service Charge", order.getServiceCharge()));
118         if (order.getDiscountAmount() > 0) {
119             sb.append(String.format("%-32s : %10.2f%n", "Discount", -order.getDiscountAmount()));
120         }
121         sb.append("================================================\n");
122         sb.append(String.format("%-32s : %10.2f%n", "GRAND TOTAL", order.getTotalAmount()));
123         sb.append("================================================\n");
124         sb.append("          THANK YOU!  VISIT US AGAIN!           \n");
125         sb.append("================================================\n");
126         return sb.toString();
127     }
128
129     /** Creates the receipts/ directory if it does not exist yet. */
```

Component	Method: buildSimpleReceipt()
Line-by-Line Breakdown	Lines 87–129 (43 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 130–137` **Method: ensureReceiptDirectoryExists()**

```
130       private void ensureReceiptDirectoryExists() throws IOException {
131           Path dir = Paths.get(RECEIPTS_DIR);
132           if (!Files.exists(dir)) {
133               Files.createDirectories(dir);
134               LOGGER.info("BillReceiptWriter: created receipts directory → " + dir.toAbsolutePath());
135           }
136       }
137   }
```

Component	Method: ensureReceiptDirectoryExists()
File Handling Logic	Interacts with the disk filesystem: ensures parent directory existence, opens output streams, and writes formatted text or CSV records.
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 130–137 (8 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

CurrencyUtil.java

Path: src\main\java\com\restaurant\util\CurrencyUtil.java | Total Lines: 22 | Logical Blocks: 5

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the CurrencyUtil role in the Restaurant Management architecture.

Lines 1–9 Package & Library Imports

```
1  package com.restaurant.util;
2
3  import com.restaurant.config.AppConfig;
4
5  import java.text.DecimalFormat;
6
7  /**
8   * Utility for formatting currency and prices consistently.
9   */
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 9. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 10–10 Class Declaration & Heritage

```
10 public class CurrencyUtil {
```

Architecture Role	Defines CurrencyUtil as a core component in the Restaurant Management System.
Utility / I/O Engine	Provides reusable, thread-safe helper services across logging, receipt serialization, and date-time arithmetic.
Access & Modifiers	Declared with public visibility on line 10, allowing cross-package invocation.

Lines 11–13 Class Members & Implementation

```
11
12     private static final DecimalFormat DECIMAL_FORMAT = new DecimalFormat("#,##0.00");
13
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 11–13 (3 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 14–18 Method: format()

```
14      public static String format(double amount) {
15          String currencySymbol = AppConfig.getInstance().getProperty("app.currency", "₹");
16          return currencySymbol + " " + DECIMAL_FORMAT.format(amount);
17      }
18
```

Component	Method: format()
Line-by-Line Breakdown	Lines 14–18 (5 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 19–22 Method: formatPlain()

```
19      public static String formatPlain(double amount) {
20          return DECIMAL_FORMAT.format(amount);
21      }
22  }
```

Component	Method: formatPlain()
Line-by-Line Breakdown	Lines 19–22 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

DateTimeUtil.java

Path: src\main\java\com\restaurant\util\DateTimeUtil.java | Total Lines: 37 | Logical Blocks: 7

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the DateTimeUtil role in the Restaurant Management architecture.

Lines 1–10 Package & Library Imports

```
1  package com.restaurant.util;
2
3  import java.time.LocalDate;
4  import java.time.LocalDateTime;
5  import java.time.LocalTime;
6  import java.time.format.DateTimeFormatter;
7
8  /**
9   * Utility for date and time formatting and calculations.
10  */
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 10. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 11–11 Class Declaration & Heritage

```
11 public class DateTimeUtil {
```

Architecture Role	Defines DateTimeUtil as a core component in the Restaurant Management System.
Utility / I/O Engine	Provides reusable, thread-safe helper services across logging, receipt serialization, and date-time arithmetic.
Access & Modifiers	Declared with public visibility on line 11, allowing cross-package invocation.

Lines 12–18 Class Members & Implementation

```
12
13      public static final DateTimeFormatter DATE_FORMATTER = DateTimeFormatter.ofPattern("dd-MM-yyyy");
14      public static final DateTimeFormatter TIME_FORMATTER = DateTimeFormatter.ofPattern("hh:mm a");
15      public static final DateTimeFormatter DATETIME_FORMATTER = DateTimeFormatter.ofPattern("dd-MM-yyyy
hh:mm a");
16      public static final DateTimeFormatter SQL_DATE_FORMATTER = DateTimeFormatter.ofPattern("yyyy-MM-d
d");
17      public static final DateTimeFormatter SQL_TIME_FORMATTER = DateTimeFormatter.ofPattern("HH:mm:s
s");
18
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 12–18 (7 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 19–23 Method: formatDate()

```
19      public static String formatDate(LocalDate date) {
20          if (date == null) return "-";
21          return date.format(DATE_FORMATTER);
22      }
23
```

Component	Method: formatDate()
Line-by-Line Breakdown	Lines 19–23 (5 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 24–28 Method: formatTime()

```
24      public static String formatTime(LocalTime time) {
25          if (time == null) return "-";
26          return time.format(TIME_FORMATTER);
27      }
28
```

Component	Method: formatTime()
Line-by-Line Breakdown	Lines 24–28 (5 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 29–33 Method: formatDateTime()

```
29      public static String formatDateTime(LocalDateTime dateTime) {
30          if (dateTime == null) return "-";
31          return dateTime.format(DATETIME_FORMATTER);
32      }
33
```

Component	Method: formatDateTime()
Line-by-Line Breakdown	Lines 29–33 (5 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 34–37 Method: getReceiptTimestamp()

```
34      public static String getReceiptTimestamp() {
35          return LocalDateTime.now().format(DATETIME_FORMATTER);
36      }
37  }
```

Component	Method: getReceiptTimestamp()
Line-by-Line Breakdown	Lines 34–37 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

PasswordUtil.java

Path: src\main\java\com\restaurant\util\PasswordUtil.java | Total Lines: 58 | Logical Blocks: 6

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the PasswordUtil role in the Restaurant Management architecture.

Lines 1–11 Package & Library Imports

```
1   package com.restaurant.util;
2
3   import java.nio.charset.StandardCharsets;
4   import java.security.MessageDigest;
5   import java.security.NoSuchAlgorithmException;
6   import java.security.SecureRandom;
7   import java.util.Base64;
8
9   /**
10   * Utility for salted SHA-256 password hashing and verification.
11   */
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 11. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 12–12 Class Declaration & Heritage

```
12  public class PasswordUtil {
```

Architecture Role	Defines PasswordUtil as a core component in the Restaurant Management System.
Utility / I/O Engine	Provides reusable, thread-safe helper services across logging, receipt serialization, and date-time arithmetic.
Access & Modifiers	Declared with public visibility on line 12, allowing cross-package invocation.

Lines 13–18 Class Members & Implementation

```
13
14      private static final SecureRandom RANDOM = new SecureRandom();
15
16      /**
17       * Generates a random cryptographic salt.
18       */
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 13–18 (6 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 19–27 Method: generateSalt()

```
19      public static String generateSalt() {
20          byte[] salt = new byte[16];
21          RANDOM.nextBytes(salt);
22          return Base64.getEncoder().encodeToString(salt);
23      }
24
25      /**
26       * Hashes plain text password with a provided salt.
27       */
```

Component	Method: generateSalt()
Line-by-Line Breakdown	Lines 19–27 (9 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 28–50 Method: hashPassword()

```
28      public static String hashPassword(String password, String salt) {
29          if (password == null) {
30              throw new IllegalArgumentException("Password cannot be null");
31          }
32          try {
33              MessageDigest md = MessageDigest.getInstance("SHA-256");
34              if (salt != null) {
35                  md.update(salt.getBytes(StandardCharsets.UTF_8));
36              }
37              byte[] hashedBytes = md.digest(password.getBytes(StandardCharsets.UTF_8));
38              StringBuilder sb = new StringBuilder();
39              for (byte b : hashedBytes) {
40                  sb.append(String.format("%02x", b));
41              }
42              return sb.toString();
43          } catch (NoSuchAlgorithmException e) {
44              throw new RuntimeException("SHA-256 algorithm not available", e);
45          }
46      }
47
48      /**
49       * Verifies plain text password against stored hash and salt.
50       */
```

Component	Method: hashPassword()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 28–50 (23 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 51–58` **Method: verifyPassword()**

```
51      public static boolean verifyPassword(String plainPassword, String storedHash, String salt) {
52          if (plainPassword == null || storedHash == null) {
53              return false;
54          }
55          String computedHash = hashPassword(plainPassword, salt);
56          return computedHash.equals(storedHash);
57      }
58  }
```

Component	Method: verifyPassword()
Line-by-Line Breakdown	Lines 51–58 (8 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

ReportExporter.java

Path: src\main\java\com\restaurant\util\ReportExporter.java | Total Lines: 175 | Logical Blocks: 9

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Disk I/O File Handling engine. Handles formatted file outputs, thread-safe append streams, and CSV/text report generation.

Lines 1–28 Package & Library Imports

```
1  package com.restaurant.util;
2
3  import com.restaurant.model.Order;
4  import com.restaurant.model.OrderItem;
5
6  import java.io.BufferedWriter;
7  import java.io.FileWriter;
8  import java.io.IOException;
9  import java.nio.file.Files;
10 import java.nio.file.Path;
11 import java.nio.file.Paths;
12 import java.time.LocalDate;
13 import java.time.format.DateTimeFormatter;
14 import java.util.List;
15 import java.util.Map;
16 import java.util.logging.Level;
17 import java.util.logging.Logger;
18
19 /**
20  * FILE HANDLING — CSV Report Exporter
21  *
22  * Exports sales and order data to CSV files in the reports/ directory.
23  * Demonstrates:
24  *   - Writing CSV-formatted text with PrintWriter/BufferedWriter
25  *   - NIO Paths/Files for directory management
26  *   - try-with-resources for safe resource cleanup
27  *   - Exception handling with meaningful error messages
28  */
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
File Handling I/O	File streams and writers (FileWriter, BufferedWriter, PrintWriter) for receipts and log generation.
Line Analysis	Covers lines 1 to 28. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 29–29 Class Declaration & Heritage

```
29   public class ReportExporter {
```

Architecture Role	Defines ReportExporter as a core component in the Restaurant Management System.
Utility / I/O Engine	Provides reusable, thread-safe helper services across logging, receipt serialization, and date-time arithmetic.
Access & Modifiers	Declared with public visibility on line 29, allowing cross-package invocation.

Lines 30–50 Class Members & Implementation

```
30
31       private static final Logger LOGGER = Logger.getLogger(ReportExporter.class.getName());
32       private static final String REPORTS_DIR = "reports";
33       private static final DateTimeFormatter DATE_FORMATTER =
34               DateTimeFormatter.ofPattern("yyyy-MM-dd");
35       private static final DateTimeFormatter TIMESTAMP_FORMATTER =
36               DateTimeFormatter.ofPattern("yyyyMMdd_HHmmss");
37
38       // ------------------------------------------------------------------------
39       // Public export methods
40       // ------------------------------------------------------------------------
41
42       /**
43        * Exports a list of orders to a CSV file.
44        * CSV columns: OrderNumber, Table, Customer, Date, Items, Total, Status
45        *
46        * @param orders   list of orders to export
47        * @param filename desired file name (without extension)
48        * @return absolute path of created file
49        * @throws IOException if the file cannot be written
50        */
```

Component	Class Members & Implementation
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 30–50 (21 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 51–91

Method: exportOrdersToCSV()

```
51      public String exportOrdersToCSV(List<Order> orders, String filename) throws IOException {
52          ensureReportsDirectoryExists();
53          String filePath = REPORTS_DIR + "/" + sanitise(filename) + ".csv";
54
55          try (BufferedWriter writer = new BufferedWriter(new FileWriter(filePath, false))) {
56              // Header row
57              writer.write("Order Number,Table,Customer,Date,Item Count,Total (INR),Status");
58              writer.newLine();
59
60              // Data rows
61              for (Order order : orders) {
62                  int itemCount = (order.getItems() != null) ? order.getItems().size() : 0;
63                  String line = String.join(",",
64                          escapeCsv(order.getOrderNumber()),
65                          escapeCsv(order.getTableNumber() != null ? order.getTableNumber() : "Takeawa
y"),
66                          escapeCsv(order.getCustomerName() != null ? order.getCustomerName() : "Walk-i
n"),
67                          escapeCsv(order.getOrderDateTime() != null
68                                  ? order.getOrderDateTime().format(DATE_FORMATTER) : ""),
69                          String.valueOf(itemCount),
70                          String.format("%.2f", order.getTotalAmount()),
71                          escapeCsv(order.getStatus() != null ? order.getStatus().name() : "")
72                  );
73                  writer.write(line);
74                  writer.newLine();
75              }
76              writer.flush();
77          }
78
79          String absolutePath = Paths.get(filePath).toAbsolutePath().toString();
80          LOGGER.info("Orders CSV exported → " + absolutePath);
81          return absolutePath;
82      }
83
84      /**
85       * Exports a category-wise sales summary to CSV.
86       * CSV columns: Category, Total Sales (INR)
87       *
88       * @param salesByCategory map of category → total sales value
89       * @return absolute path of created file
90       * @throws IOException if the file cannot be written
91       */
```

Component	Method: exportOrdersToCSV()
File Handling Logic	Interacts with the disk filesystem: ensures parent directory existence, opens output streams, and writes formatted text or CSV records.
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 51–91 (41 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 92–120 Method: exportSalesByCategoryCSV()

```
92      public String exportSalesByCategoryCSV(Map<String, Double> salesByCategory) throws IOException {
93          ensureReportsDirectoryExists();
94          String timestamp = java.time.LocalDateTime.now().format(TIMESTAMP_FORMATTER);
95          String filePath  = REPORTS_DIR + "/sales_by_category_" + timestamp + ".csv";
96
97          try (BufferedWriter writer = new BufferedWriter(new FileWriter(filePath, false))) {
98              writer.write("Category,Total Sales (INR)");
99              writer.newLine();
100             for (Map.Entry<String, Double> entry : salesByCategory.entrySet()) {
101                 writer.write(escapeCsv(entry.getKey()) + "," +
102                              String.format("%.2f", entry.getValue()));
103                 writer.newLine();
104             }
105             writer.flush();
106         }
107
108         String absolutePath = Paths.get(filePath).toAbsolutePath().toString();
109         LOGGER.info("Sales-by-category CSV exported → " + absolutePath);
110         return absolutePath;
111     }
112
113     /**
114      * Exports today's detailed order items to CSV — useful for kitchen/manager review.
115      * CSV columns: OrderNumber, ItemName, Quantity, UnitPrice, Subtotal
116      *
117      * @param orders list of today's orders
118      * @return absolute path of created file
119      * @throws IOException if the file cannot be written
120      */
```

Component	Method: exportSalesByCategoryCSV()
File Handling Logic	Interacts with the disk filesystem: ensures parent directory existence, opens output streams, and writes formatted text or CSV records.
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 92–120 (29 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 121–156 Method: exportDailyItemsCSV()

```
121     public String exportDailyItemsCSV(List<Order> orders) throws IOException {
122         ensureReportsDirectoryExists();
123         String today    = LocalDate.now().format(DATE_FORMATTER);
124         String filePath = REPORTS_DIR + "/daily_items_" + today + ".csv";
125 
126         try (BufferedWriter writer = new BufferedWriter(new FileWriter(filePath, false))) {
127             writer.write("Order Number,Item Name,Quantity,Unit Price (INR),Subtotal (INR)");
128             writer.newLine();
129 
130             for (Order order : orders) {
131                 if (order.getItems() == null) continue;
132                 for (OrderItem item : order.getItems()) {
133                     String line = String.join(",",
134                             escapeCsv(order.getOrderNumber()),
135                             escapeCsv(item.getItemName()),
136                             String.valueOf(item.getQuantity()),
137                             String.format("%.2f", item.getUnitPrice()),
138                             String.format("%.2f", item.getSubtotal())
139                     );
140                     writer.write(line);
141                     writer.newLine();
142                 }
143             }
144             writer.flush();
145         }
146 
147         String absolutePath = Paths.get(filePath).toAbsolutePath().toString();
148         LOGGER.info("Daily items CSV exported → " + absolutePath);
149         return absolutePath;
150     }
151 
152     // ------------------------------------------------------------------------
153     // Helpers
154     // ------------------------------------------------------------------------
155 
156     /** Creates the reports/ directory if it does not exist. */
```

Component	Method: exportDailyItemsCSV()
File Handling Logic	Interacts with the disk filesystem: ensures parent directory existence, opens output streams, and writes formatted text or CSV records.
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 121–156 (36 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 157–165` **Method: ensureReportsDirectoryExists()**

```
157       private void ensureReportsDirectoryExists() throws IOException {
158           Path dir = Paths.get(REPORTS_DIR);
159           if (!Files.exists(dir)) {
160               Files.createDirectories(dir);
161               LOGGER.info("ReportExporter: created reports directory → " + dir.toAbsolutePath());
162           }
163       }
164
165       /** Wraps a field in double quotes and escapes any internal double quotes. */
```

Component	Method: ensureReportsDirectoryExists()
File Handling Logic	Interacts with the disk filesystem: ensures parent directory existence, opens output streams, and writes formatted text or CSV records.
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 157–165 (9 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 166–171` **Method: escapeCsv()**

```
166       private String escapeCsv(String value) {
167           if (value == null) return "\"\"";
168           return "\"" + value.replace("\"", "\"\"") + "\"";
169       }
170
171       /** Removes characters that are unsafe in file names. */
```

Component	Method: escapeCsv()
Line-by-Line Breakdown	Lines 166–171 (6 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 172–175` **Method: sanitise()**

```
172       private String sanitise(String name) {
173           return name.replaceAll("[^a-zA-Z0-9_\\-]", "_");
174       }
175   }
```

Component	Method: sanitise()
Line-by-Line Breakdown	Lines 172–175 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

ValidationUtil.java

Path: src\main\java\com\restaurant\util\ValidationUtil.java | Total Lines: 58 | Logical Blocks: 13

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the ValidationUtil role in the Restaurant Management architecture.

Lines 1–7 Package & Library Imports

```
1  package com.restaurant.util;
2
3  import java.util.regex.Pattern;
4
5  /**
6   * Utility for input validation and sanitization.
7   */
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 7. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 8–8 Class Declaration & Heritage

```
8  public class ValidationUtil {
```

Architecture Role	Defines ValidationUtil as a core component in the Restaurant Management System.
Utility / I/O Engine	Provides reusable, thread-safe helper services across logging, receipt serialization, and date-time arithmetic.
Access & Modifiers	Declared with public visibility on line 8, allowing cross-package invocation.

Lines 9–12 Class Members & Implementation

```
9
10      private static final Pattern EMAIL_PATTERN = Pattern.compile("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$");
11      private static final Pattern PHONE_PATTERN = Pattern.compile("^[+]?[0-9\\s-]{7,15}$");
12
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 9–12 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 13–16 Method: isNotEmpty()

```
13      public static boolean isNotEmpty(String str) {
14          return str != null && !str.trim().isEmpty();
15      }
16
```

Component	Method: isNotEmpty()
Line-by-Line Breakdown	Lines 13–16 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 17–23 Method: isValidEmail()

```
17      public static boolean isValidEmail(String email) {
18          if (email == null || email.trim().isEmpty()) {
19              return false;
20          }
21          return EMAIL_PATTERN.matcher(email.trim()).matches();
22      }
23
```

Component	Method: isValidEmail()
Line-by-Line Breakdown	Lines 17–23 (7 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 24–30 Method: isValidPhone()

```
24      public static boolean isValidPhone(String phone) {
25          if (phone == null || phone.trim().isEmpty()) {
26              return false;
27          }
28          return PHONE_PATTERN.matcher(phone.trim()).matches();
29      }
30
```

Component	Method: isValidPhone()
Line-by-Line Breakdown	Lines 24–30 (7 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 31–34 Method: isPositive()

```
31      public static boolean isPositive(double value) {
32          return value > 0;
33      }
34
```

Component	Method: isPositive()
Line-by-Line Breakdown	Lines 31–34 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 35–38 Method: isNonNegative()

```
35      public static boolean isNonNegative(double value) {
36          return value >= 0;
37      }
38
```

Component	Method: isNonNegative()
Line-by-Line Breakdown	Lines 35–38 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 39–42 Method: isPositive()

```
39      public static boolean isPositive(int value) {
40          return value > 0;
41      }
42
```

Component	Method: isPositive()
Line-by-Line Breakdown	Lines 39–42 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 43–46 Method: isNonNegative()

```
43      public static boolean isNonNegative(int value) {
44          return value >= 0;
45      }
46
```

Component	Method: isNonNegative()
Line-by-Line Breakdown	Lines 43–46 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 47–50 Method: isValidPrice()

```
47      public static boolean isValidPrice(double price) {
48          return price >= 0;
49      }
50
```

Component	Method: isValidPrice()
Line-by-Line Breakdown	Lines 47–50 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 51–54` Method: isValidCapacity()

```
51      public static boolean isValidCapacity(int capacity) {
52          return capacity > 0 && capacity <= 100;
53      }
54
```

Component	Method: isValidCapacity()
Line-by-Line Breakdown	Lines 51–54 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 55–58` Method: isValidDiscount()

```
55      public static boolean isValidDiscount(double discount, double subtotal) {
56          return discount >= 0 && discount <= subtotal;
57      }
58  }
```

Component	Method: isValidDiscount()
Line-by-Line Breakdown	Lines 55–58 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

DatabaseOperationException.java

Path: src\main\java\com\restaurant\exception\DatabaseOperationException.java | Total Lines: 11 | Logical Blocks: 3

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the DatabaseOperationException role in the Restaurant Management architecture.

Lines 1–2 Package & Library Imports

```
1   package com.restaurant.exception;
2
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 2. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 3–3 Class Declaration & Heritage

```
3   public class DatabaseOperationException extends RuntimeException {
```

Architecture Role	Defines DatabaseOperationException as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 3, allowing cross-package invocation.

Lines 4–11 Class Members & Implementation

```
4       public DatabaseOperationException(String message) {
5           super(message);
6       }
7
8       public DatabaseOperationException(String message, Throwable cause) {
9           super(message, cause);
10      }
11  }
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 4–11 (8 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

InsufficientStockException.java

Path: src\main\java\com\restaurant\exception\InsufficientStockException.java | Total Lines: 7 | Logical Blocks: 3

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the InsufficientStockException role in the Restaurant Management architecture.

Lines 1–2 Package & Library Imports

```
1  package com.restaurant.exception;
2
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 2. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 3–3 Class Declaration & Heritage

```
3  public class InsufficientStockException extends Exception {
```

Architecture Role	Defines InsufficientStockException as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 3, allowing cross-package invocation.

Lines 4–7 Class Members & Implementation

```
4      public InsufficientStockException(String message) {
5          super(message);
6      }
7  }
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 4–7 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

InvalidLoginException.java

Path: src\main\java\com\restaurant\exception\InvalidLoginException.java | Total Lines: 7 | Logical Blocks: 3

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the InvalidLoginException role in the Restaurant Management architecture.

Lines 1–2 Package & Library Imports

```
1  package com.restaurant.exception;
2
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 2. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 3–3 Class Declaration & Heritage

```
3  public class InvalidLoginException extends Exception {
```

Architecture Role	Defines InvalidLoginException as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 3, allowing cross-package invocation.

Lines 4–7 Class Members & Implementation

```
4      public InvalidLoginException(String message) {
5          super(message);
6      }
7  }
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 4–7 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

InvalidOrderException.java

Path: src\main\java\com\restaurant\exception\InvalidOrderException.java | Total Lines: 7 | Logical Blocks: 3

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the InvalidOrderException role in the Restaurant Management architecture.

Lines 1–2 Package & Library Imports

```
1  package com.restaurant.exception;
2
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 2. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 3–3 Class Declaration & Heritage

```
3  public class InvalidOrderException extends Exception {
```

Architecture Role	Defines InvalidOrderException as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 3, allowing cross-package invocation.

Lines 4–7 Class Members & Implementation

```
4      public InvalidOrderException(String message) {
5          super(message);
6      }
7  }
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 4–7 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

InvalidReservationException.java

Path: src\main\java\com\restaurant\exception\InvalidReservationException.java | Total Lines: 7 | Logical Blocks: 3

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the InvalidReservationException role in the Restaurant Management architecture.

Lines 1–2 Package & Library Imports

```
1  package com.restaurant.exception;
2
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 2. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 3–3 Class Declaration & Heritage

```
3  public class InvalidReservationException extends Exception {
```

Architecture Role	Defines InvalidReservationException as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 3, allowing cross-package invocation.

Lines 4–7 Class Members & Implementation

```
4      public InvalidReservationException(String message) {
5          super(message);
6      }
7  }
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 4–7 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

MenuItemNotAvailableException.java

Path: src\main\java\com\restaurant\exception\MenuItemNotAvailableException.java | Total Lines: 7 | Logical Blocks: 3

📘 COMPONENT OVERVIEW & RESPONSIBILITY

Defines and executes the MenuItemNotAvailableException role in the Restaurant Management architecture.

Lines 1–2 Package & Library Imports

```
1  package com.restaurant.exception;
2
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 2. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 3–3 Class Declaration & Heritage

```
3  public class MenuItemNotAvailableException extends Exception {
```

Architecture Role	Defines MenuItemNotAvailableException as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 3, allowing cross-package invocation.

Lines 4–7 Class Members & Implementation

```
4      public MenuItemNotAvailableException(String message) {
5          super(message);
6      }
7  }
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 4–7 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

PaymentFailedException.java

Path: src\main\java\com\restaurant\exception\PaymentFailedException.java | Total Lines: 7 | Logical Blocks: 3

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the PaymentFailedException role in the Restaurant Management architecture.

Lines 1–2 Package & Library Imports

```
1  package com.restaurant.exception;
2
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 2. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 3–3 Class Declaration & Heritage

```
3  public class PaymentFailedException extends Exception {
```

Architecture Role	Defines PaymentFailedException as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 3, allowing cross-package invocation.

Lines 4–7 Class Members & Implementation

```
4      public PaymentFailedException(String message) {
5          super(message);
6      }
7  }
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 4–7 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

TableNotAvailableException.java

Path: src\main\java\com\restaurant\exception\TableNotAvailableException.java | Total Lines: 7 | Logical Blocks: 3

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the TableNotAvailableException role in the Restaurant Management architecture.

Lines 1–2 Package & Library Imports

```
1  package com.restaurant.exception;
2
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 2. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 3–3 Class Declaration & Heritage

```
3  public class TableNotAvailableException extends Exception {
```

Architecture Role	Defines TableNotAvailableException as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 3, allowing cross-package invocation.

Lines 4–7 Class Members & Implementation

```
4      public TableNotAvailableException(String message) {
5          super(message);
6      }
7  }
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 4–7 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

DataStructureManager.java

Path: src\main\java\com\restaurant\data\DataStructureManager.java | Total Lines: 82 | Logical Blocks: 11

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the DataStructureManager role in the Restaurant Management architecture.

Lines 1–15 Package & Library Imports

```
1   package com.restaurant.data;
2
3   import com.restaurant.model.Customer;
4   import com.restaurant.model.MenuItem;
5   import com.restaurant.model.Order;
6
7   import java.util.*;
8
9   /**
10   * Demonstrates centralized application state management utilizing diverse Java Collections:
11   * - HashMap for fast O(1) key-based lookups.
12   * - HashSet for maintaining distinct unique active sessions/customers.
13   * - ArrayList for flexible indexed item sequences.
14   * - PriorityQueue for kitchen preparation dispatch.
15   */
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 15. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 16–16 Class Declaration & Heritage

```
16  public class DataStructureManager {
```

Architecture Role	Defines DataStructureManager as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 16, allowing cross-package invocation.

Lines 17–39 Class Members & Implementation

```
17      private static DataStructureManager instance;
18
19      // Fast O(1) in-memory item lookup cache
20      private final Map<Integer, MenuItem> menuCache = new HashMap<>();
21
22      // Fast unique phone lookup for customers
23      private final Map<String, Customer> customerPhoneMap = new HashMap<>();
24
25      // Set of active logged-in usernames
26      private final Set<String> activeUserSessions = new HashSet<>();
27
28      // Kitchen dispatch queue
29      private final OrderQueue kitchenOrderQueue = new OrderQueue();
30
31      private DataStructureManager() {}
32
33      public static synchronized DataStructureManager getInstance() {
34          if (instance == null) {
35              instance = new DataStructureManager();
36          }
37          return instance;
38      }
39
```

Component	Class Members & Implementation
Thread Synchronization	Guarantees thread-safety: prevents race conditions by allowing only one thread to execute this critical block at any instant.
Line-by-Line Breakdown	Lines 17–39 (23 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 40–48 Method: cacheMenuItems()

```
40      public void cacheMenuItems(List<MenuItem> items) {
41          menuCache.clear();
42          if (items != null) {
43              for (MenuItem item : items) {
44                  menuCache.put(item.getItemId(), item);
45              }
46          }
47      }
48
```

Component	Method: cacheMenuItems()
Line-by-Line Breakdown	Lines 40–48 (9 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 49–52 Method: getMenuItemById()

```
49      public MenuItem getMenuItemById(int itemId) {
50          return menuCache.get(itemId);
51      }
52
```

Component	Method: getMenuItemById()
Line-by-Line Breakdown	Lines 49–52 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 53–58 Method: registerCustomerInIndex()

```
53      public void registerCustomerInIndex(Customer customer) {
54          if (customer != null && customer.getPhone() != null) {
55              customerPhoneMap.put(customer.getPhone(), customer);
56          }
57      }
58
```

Component	Method: registerCustomerInIndex()
Line-by-Line Breakdown	Lines 53–58 (6 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 59–62 Method: findCustomerByPhone()

```
59      public Customer findCustomerByPhone(String phone) {
60          return customerPhoneMap.get(phone);
61      }
62
```

Component	Method: findCustomerByPhone()
Line-by-Line Breakdown	Lines 59–62 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 63–68 Method: registerSession()

```
63      public void registerSession(String username) {
64          if (username != null) {
65              activeUserSessions.add(username);
66          }
67      }
68
```

Component	Method: registerSession()
Line-by-Line Breakdown	Lines 63–68 (6 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 69–74 Method: endSession()

```
69      public void endSession(String username) {
70          if (username != null) {
71              activeUserSessions.remove(username);
72          }
73      }
74
```

Component	Method: endSession()
Line-by-Line Breakdown	Lines 69–74 (6 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 75–78 Method: isUserActive()

```
75      public boolean isUserActive(String username) {
76          return activeUserSessions.contains(username);
77      }
78
```

Component	Method: isUserActive()
Line-by-Line Breakdown	Lines 75–78 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 79–82 Method: getKitchenOrderQueue()

```
79      public OrderQueue getKitchenOrderQueue() {
80          return kitchenOrderQueue;
81      }
82  }
```

Component	Method: getKitchenOrderQueue()
Line-by-Line Breakdown	Lines 79–82 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

OrderQueue.java

Path: src\main\java\com\restaurant\data\OrderQueue.java | Total Lines: 95 | Logical Blocks: 8

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the OrderQueue role in the Restaurant Management architecture.

Lines 1–10 Package & Library Imports

```
1  package com.restaurant.data;
2
3  import com.restaurant.model.Order;
4
5  import java.util.*;
6
7  /**
8   * Custom Queue manager utilizing Java's PriorityQueue and LinkedList Queue.
9   * Prioritizes orders in the kitchen based on wait time and VIP/urgency criteria.
10  */
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 10. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 11–11 Class Declaration & Heritage

```
11  public class OrderQueue {
```

Architecture Role	Defines OrderQueue as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 11, allowing cross-package invocation.

Lines 12–26 Class Members & Implementation

```
12
13      /**
14       * Kitchen ticket wrapper holding priority scoring.
15       */
16      public static class KitchenTicket implements Comparable<KitchenTicket> {
17          private final Order order;
18          private final long queueEntryTimestamp;
19          private final int priorityScore; // Higher value = higher priority
20
21          public KitchenTicket(Order order, int priorityScore) {
22              this.order = order;
23              this.queueEntryTimestamp = System.currentTimeMillis();
24              this.priorityScore = priorityScore;
25          }
26
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 12–26 (15 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 27–30 Method: getOrder()

```
27          public Order getOrder() {
28              return order;
29          }
30
```

Component	Method: getOrder()
Line-by-Line Breakdown	Lines 27–30 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 31–34 Method: getQueueEntryTimestamp()

```
31          public long getQueueEntryTimestamp() {
32              return queueEntryTimestamp;
33          }
34
```

Component	Method: getQueueEntryTimestamp()
Line-by-Line Breakdown	Lines 31–34 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 35–39` Method: getPriorityScore()

```
35          public int getPriorityScore() {
36              return priorityScore;
37          }
38
39          @Override
```

Component	Method: getPriorityScore()
Line-by-Line Breakdown	Lines 35–39 (5 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 40–85 **Method: compareTo()**

```
40          public int compareTo(KitchenTicket other) {
41              // Higher priority score first, then older queue entry timestamp (FIFO tie breaker)
42              if (this.priorityScore != other.priorityScore) {
43                  return Integer.compare(other.priorityScore, this.priorityScore);
44              }
45              return Long.compare(this.queueEntryTimestamp, other.queueEntryTimestamp);
46          }
47      }
48
49      private final PriorityQueue<KitchenTicket> kitchenPriorityQueue;
50      private final Queue<Order> regularFifoQueue;
51
52      public OrderQueue() {
53          this.kitchenPriorityQueue = new PriorityQueue<>();
54          this.regularFifoQueue = new LinkedList<>();
55      }
56
57      public synchronized void enqueue(Order order, int priority) {
58          if (order != null) {
59              kitchenPriorityQueue.offer(new KitchenTicket(order, priority));
60              regularFifoQueue.offer(order);
61          }
62      }
63
64      public synchronized Order dequeuePriority() {
65          KitchenTicket ticket = kitchenPriorityQueue.poll();
66          if (ticket != null) {
67              regularFifoQueue.remove(ticket.getOrder());
68              return ticket.getOrder();
69          }
70          return null;
71      }
72
73      public synchronized List<Order> getAllPendingPriorityOrders() {
74          List<KitchenTicket> copy = new ArrayList<>(kitchenPriorityQueue);
75          Collections.sort(copy);
76          List<Order> orderedList = new ArrayList<>();
77          for (KitchenTicket ticket : copy) {
78              orderedList.add(ticket.getOrder());
79          }
80          return orderedList;
81      }
82
83      public synchronized int size() {
84          return kitchenPriorityQueue.size();
85      }
```

Component	Method: compareTo()
Thread Synchronization	Guarantees thread-safety: prevents race conditions by allowing only one thread to execute this critical block at any instant.
Line-by-Line Breakdown	Lines 40–85 (46 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 86–95` Class Members & Continuation

```
86
87      public synchronized boolean isEmpty() {
88          return kitchenPriorityQueue.isEmpty();
89      }
90
91      public synchronized void clear() {
92          kitchenPriorityQueue.clear();
93          regularFifoQueue.clear();
94      }
95  }
```

Component	Class Members & Continuation
Thread Synchronization	Guarantees thread-safety: prevents race conditions by allowing only one thread to execute this critical block at any instant.
Line-by-Line Breakdown	Lines 86–95 (10 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

SearchSortUtil.java

Path: src\main\java\com\restaurant\data\SearchSortUtil.java | Total Lines: 122 | Logical Blocks: 11

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the SearchSortUtil role in the Restaurant Management architecture.

Lines 1–11 Package & Library Imports

```
1   package com.restaurant.data;
2
3   import com.restaurant.model.InventoryItem;
4   import com.restaurant.model.MenuItem;
5
6   import java.util.ArrayList;
7   import java.util.List;
8
9   /**
10   * Utility algorithms demonstrating Binary Search and QuickSort implementations.
11   */
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
Line Analysis	Covers lines 1 to 11. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 12–12 Class Declaration & Heritage

```
12  public class SearchSortUtil {
```

Architecture Role	Defines SearchSortUtil as a core component in the Restaurant Management System.
Utility / I/O Engine	Provides reusable, thread-safe helper services across logging, receipt serialization, and date-time arithmetic.
Access & Modifiers	Declared with public visibility on line 12, allowing cross-package invocation.

Lines 13–17 Class Members & Implementation

```
13
14      /**
15       * Binary search on a list of menu items sorted by price.
16       * @return index of found item, or -1 if not found.
17       */
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 13–17 (5 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 18–41 Method: binarySearchMenuByPrice()

```
18      public static int binarySearchMenuByPrice(List<MenuItem> sortedList, double targetPrice) {
19          if (sortedList == null || sortedList.isEmpty()) return -1;
20          int low = 0;
21          int high = sortedList.size() - 1;
22
23          while (low <= high) {
24              int mid = low + (high - low) / 2;
25              double midVal = sortedList.get(mid).getPrice();
26
27              if (Math.abs(midVal - targetPrice) < 0.001) {
28                  return mid;
29              }
30              if (midVal < targetPrice) {
31                  low = mid + 1;
32              } else {
33                  high = mid - 1;
34              }
35          }
36          return -1;
37      }
38
39      /**
40       * In-place QuickSort algorithm for MenuItem list by price ascending.
41       */
```

Component	Method: binarySearchMenuByPrice()
Line-by-Line Breakdown	Lines 18–41 (24 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 42–46 Method: quickSortMenuItemsByPrice()

```
42      public static void quickSortMenuItemsByPrice(List<MenuItem> items) {
43          if (items == null || items.size() <= 1) return;
44          quickSortMenuHelper(items, 0, items.size() - 1);
45      }
46
```

Component	Method: quickSortMenuItemsByPrice()
Line-by-Line Breakdown	Lines 42–46 (5 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 47–54 Method: quickSortMenuHelper()

```
47      private static void quickSortMenuHelper(List<MenuItem> items, int low, int high) {
48          if (low < high) {
49              int pivotIndex = partitionMenu(items, low, high);
50              quickSortMenuHelper(items, low, pivotIndex - 1);
51              quickSortMenuHelper(items, pivotIndex + 1, high);
52          }
53      }
54
```

Component	Method: quickSortMenuHelper()
Line-by-Line Breakdown	Lines 47–54 (8 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 55–74 Method: partitionMenu()

```
55      private static int partitionMenu(List<MenuItem> items, int low, int high) {
56          double pivot = items.get(high).getPrice();
57          int i = low - 1;
58          for (int j = low; j < high; j++) {
59              if (items.get(j).getPrice() <= pivot) {
60                  i++;
61                  MenuItem temp = items.get(i);
62                  items.set(i, items.get(j));
63                  items.set(j, temp);
64              }
65          }
66          MenuItem temp = items.get(i + 1);
67          items.set(i + 1, items.get(high));
68          items.set(high, temp);
69          return i + 1;
70      }
71
72      /**
73       * QuickSort for Inventory items by stock quantity ascending (to bring low-stock to top).
74       */
```

Component	Method: partitionMenu()
Line-by-Line Breakdown	Lines 55–74 (20 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 75–79 Method: quickSortInventoryByQuantity()

```
75      public static void quickSortInventoryByQuantity(List<InventoryItem> items) {
76          if (items == null || items.size() <= 1) return;
77          quickSortInventoryHelper(items, 0, items.size() - 1);
78      }
79
```

Component	Method: quickSortInventoryByQuantity()
Line-by-Line Breakdown	Lines 75–79 (5 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 80–87 Method: quickSortInventoryHelper()

```
80      private static void quickSortInventoryHelper(List<InventoryItem> items, int low, int high) {
81          if (low < high) {
82              int pivotIndex = partitionInventory(items, low, high);
83              quickSortInventoryHelper(items, low, pivotIndex - 1);
84              quickSortInventoryHelper(items, pivotIndex + 1, high);
85          }
86      }
87
```

Component	Method: quickSortInventoryHelper()
Line-by-Line Breakdown	Lines 80–87 (8 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 88–107 Method: partitionInventory()

```
88      private static int partitionInventory(List<InventoryItem> items, int low, int high) {
89          double pivot = items.get(high).getQuantity();
90          int i = low - 1;
91          for (int j = low; j < high; j++) {
92              if (items.get(j).getQuantity() <= pivot) {
93                  i++;
94                  InventoryItem temp = items.get(i);
95                  items.set(i, items.get(j));
96                  items.set(j, temp);
97              }
98          }
99          InventoryItem temp = items.get(i + 1);
100         items.set(i + 1, items.get(high));
101         items.set(high, temp);
102         return i + 1;
103     }
104
105     /**
106      * Linear search filter for names containing query (case-insensitive).
107      */
```

Component	Method: partitionInventory()
Line-by-Line Breakdown	Lines 88–107 (20 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

`Lines 108–122` **Method: filterMenuByName()**

```
108       public static List<MenuItem> filterMenuByName(List<MenuItem> items, String query) {
109           List<MenuItem> results = new ArrayList<>();
110           if (items == null) return results;
111           if (query == null || query.trim().isEmpty()) return new ArrayList<>(items);
112
113           String lowerQuery = query.toLowerCase().trim();
114           for (MenuItem item : items) {
115               if (item.getName().toLowerCase().contains(lowerQuery) ||
116                   (item.getDescription() != null && item.getDescription().toLowerCase().contains(lowerQu
ery))) {
117                   results.add(item);
118               }
119           }
120           return results;
121       }
122   }
```

Component	Method: filterMenuByName()
Line-by-Line Breakdown	Lines 108–122 (15 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Main.java

Path: src\main\java\com\restaurant\Main.java | Total Lines: 155 | Logical Blocks: 10

> 📘 **COMPONENT OVERVIEW & RESPONSIBILITY**
>
> Defines and executes the Main role in the Restaurant Management architecture.

Lines 1–29 Package & Library Imports

```
1   package com.restaurant;
2
3   import com.restaurant.config.DatabaseConfig;
4   import com.restaurant.service.AsyncReportGenerator;
5   import com.restaurant.service.DatabaseHealthMonitor;
6   import com.restaurant.service.KitchenOrderProcessor;
7   import com.restaurant.util.ActivityLogger;
8   import javafx.application.Application;
9   import javafx.application.Platform;
10  import javafx.fxml.FXMLLoader;
11  import javafx.scene.Parent;
12  import javafx.scene.Scene;
13  import javafx.stage.Stage;
14
15  import java.util.logging.Level;
16  import java.util.logging.Logger;
17
18  /**
19   * Main Application Launcher for Restaurant Management System.
20   *
21   * Integrates three advanced Java concepts:
22   *  1. EXCEPTION HANDLING  – global uncaught exception handler catches any
23   *                           unhandled RuntimeException on any thread and logs it.
24   *  2. MULTI-THREADING     – starts KitchenOrderProcessor (background daemon thread)
25   *                           and DatabaseHealthMonitor (ScheduledExecutorService) on launch;
26   *                           stops them cleanly in stop().
27   *  3. FILE HANDLING       – ActivityLogger records every major lifecycle event to
28   *                           logs/activity.log using BufferedWriter.
29   */
```

Purpose	Defines the namespace package and imports essential Java standard libraries (collections, concurrency, I/O, SQL) and JavaFX UI components.
JavaFX Dependency	Binds FXML controls (TableViews, Buttons, ComboBoxes, Labels) to enable real-time UI interaction.
Database Integration	Imports JDBC driver contracts for executing SQL statements and connection pooling.
Line Analysis	Covers lines 1 to 29. Cleanly separates dependencies without wildcard star imports where specificity is required.

Lines 30–30 Class Declaration & Heritage

```
30   public class Main extends Application {
```

Architecture Role	Defines Main as a core component in the Restaurant Management System.
Domain Entity	Encapsulates business entity attributes, immutable identity, and state transformations.
Access & Modifiers	Declared with public visibility on line 30, allowing cross-package invocation.

Lines 31–39 Class Members & Implementation

```
31
32       private static final Logger LOGGER = Logger.getLogger(Main.class.getName());
33       private static Stage primaryStage;
34
35       // ------------------------------------------------------------------------
36       // JavaFX Application lifecycle
37       // ------------------------------------------------------------------------
38
39       @Override
```

Component	Class Members & Implementation
Line-by-Line Breakdown	Lines 31–39 (9 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 40–61

Method: init()

```
40      public void init() {
41          // EXCEPTION HANDLING — install a global handler that catches any
42          // uncaught exception on ANY thread (UI, worker, scheduled, etc.)
43          Thread.setDefaultUncaughtExceptionHandler((thread, throwable) -> {
44              LOGGER.log(Level.SEVERE,
45                      "Uncaught exception on thread [" + thread.getName() + "]", throwable);
46              ActivityLogger.getInstance().logError(
47                      "UncaughtException[" + thread.getName() + "]",
48                      throwable.getMessage() != null ? throwable.getMessage() : throwable.getClass().get
Name());
49          });
50
51          LOGGER.info("Global uncaught exception handler installed.");
52
53          // MULTI-THREADING — start background services before the UI appears
54          KitchenOrderProcessor.getInstance().start();      // daemon worker thread
55          DatabaseHealthMonitor.getInstance().startMonitoring(); // scheduled pinger
56
57          // FILE HANDLING — log application startup
58          ActivityLogger.getInstance().logInfo("Application started — Grand Feast RMS v1.0");
59      }
60
61      @Override
```

Component	Method: init()
Multi-Threading Logic	Manages worker threads, background queues, or concurrent task execution without blocking the JavaFX UI rendering loop.
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 40–61 (22 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 62–92 **Method: start()**

```
62      public void start(Stage stage) {
63          primaryStage = stage;
64          primaryStage.setTitle("Grand Feast — Restaurant Management System");
65
66          // Verify DB connectivity in background (MULTI-THREADING: anonymous Thread)
67          new Thread(() -> {
68              try {
69                  boolean connected = DatabaseConfig.getInstance().testConnection();
70                  if (connected) {
71                      LOGGER.info("Successfully connected to MySQL database on startup.");
72                      ActivityLogger.getInstance().logInfo("MySQL connection verified on startup.");
73                  } else {
74                      LOGGER.warning("Could not connect to MySQL. Verify credentials in application.prop
erties.");
75                      ActivityLogger.getInstance().logError("Startup", "MySQL connection test failed.");
76                  }
77              } catch (Exception e) {
78                  // EXCEPTION HANDLING — catch DB connection failure gracefully
79                  LOGGER.log(Level.SEVERE, "Exception during startup DB check", e);
80                  ActivityLogger.getInstance().logError("Startup DB check", e.getMessage());
81              }
82          }, "Startup-DB-Check-Thread").start();
83
84          loadLoginScreen();
85      }
86
87      /**
88       * Called by JavaFX when the application is closing.
89       * MULTI-THREADING: gracefully shuts down all background threads/executors.
90       * FILE HANDLING: logs the shutdown event.
91       */
92      @Override
```

Component	Method: start()
Multi-Threading Logic	Manages worker threads, background queues, or concurrent task execution without blocking the JavaFX UI rendering loop.
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 62–92 (31 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 93–110 Method: stop()

```
93      public void stop() {
94          LOGGER.info("Application shutting down — stopping background services...");
95
96          // Shut down multi-threaded services cleanly
97          KitchenOrderProcessor.getInstance().stop();
98          DatabaseHealthMonitor.getInstance().stopMonitoring();
99          AsyncReportGenerator.getInstance().shutdown();
100
101         // FILE HANDLING — write shutdown event to log
102         ActivityLogger.getInstance().logInfo("Application stopped — all services shut down.");
103
104         LOGGER.info("All background services stopped. Goodbye.");
105     }
106
107     // ------------------------------------------------------------------------
108     // Screen navigation helpers
109     // ------------------------------------------------------------------------
110
```

Component	Method: stop()
Line-by-Line Breakdown	Lines 93–110 (18 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 111–128 Method: loadLoginScreen()

```
111     public static void loadLoginScreen() {
112         try {
113             FXMLLoader loader = new FXMLLoader(Main.class.getResource("/fxml/login.fxml"));
114             Parent root = loader.load();
115             Scene scene = new Scene(root, 900, 600);
116             scene.getStylesheets().add(
117                     Main.class.getResource("/css/application.css").toExternalForm());
118             primaryStage.setScene(scene);
119             primaryStage.setResizable(false);
120             primaryStage.centerOnScreen();
121             primaryStage.show();
122         } catch (Exception e) {
123             // EXCEPTION HANDLING — log and prevent silent failure
124             LOGGER.log(Level.SEVERE, "Failed to load login screen", e);
125             ActivityLogger.getInstance().logError("loadLoginScreen", e.getMessage());
126         }
127     }
128
```

Component	Method: loadLoginScreen()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 111–128 (18 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 129–146 Method: loadMainDashboard()

```
129       public static void loadMainDashboard() {
130           try {
131               FXMLLoader loader = new FXMLLoader(Main.class.getResource("/fxml/dashboard.fxml"));
132               Parent root = loader.load();
133               Scene scene = new Scene(root, 1280, 800);
134               scene.getStylesheets().add(
135                       Main.class.getResource("/css/application.css").toExternalForm());
136               primaryStage.setScene(scene);
137               primaryStage.setResizable(true);
138               primaryStage.centerOnScreen();
139               primaryStage.show();
140           } catch (Exception e) {
141               // EXCEPTION HANDLING — log and prevent silent failure
142               LOGGER.log(Level.SEVERE, "Failed to load dashboard screen", e);
143               ActivityLogger.getInstance().logError("loadMainDashboard", e.getMessage());
144           }
145       }
146
```

Component	Method: loadMainDashboard()
Exception Handling	Implements robust error handling (try-with-resources or try-catch). Automatically closes database or file stream handles, eliminating resource leaks.
Line-by-Line Breakdown	Lines 129–146 (18 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 147–150 Method: getPrimaryStage()

```
147       public static Stage getPrimaryStage() {
148           return primaryStage;
149       }
150
```

Component	Method: getPrimaryStage()
Line-by-Line Breakdown	Lines 147–150 (4 lines): Implements operational logic, field assignments, validation checks, and event dispatches.

Lines 151–155 Method: main()

```
151       public static void main(String[] args) {
152           launch(args);
153       }
154   }
155
```

Component	Method: main()
Line-by-Line Breakdown	Lines 151–155 (5 lines): Implements operational logic, field assignments, validation checks, and event dispatches.